



UniMe
1548

UNIVERSITY OF MESSINA

Theme Development with WordPress

Web Programming Project

Prof. Armando Ruggeri

Students:

-Aditi Dole

-Meet Tagadiya

Introduction:

Theme development in WordPress involves creating custom designs and layouts for websites. These themes control how a site looks and works, including its layout, colors, fonts, and more. Themes are like the foundation of a WordPress site, giving it a cohesive and attractive appearance.

While WordPress offers many ready-made themes, custom development lets site owners personalize their sites. This process includes coding in languages like HTML, CSS, JavaScript, SQL, and PHP to build unique themes from scratch.

In researching this topic, we found a helpful YouTube tutorial that provided insights into the workings of theme development in WordPress, offering practical guidance on how to get started and showcasing best practices in design and development. The tutorial was useful for understanding theme development better and why it's important for making websites that people enjoy using.

Objective:

The primary objective of this WordPress theme development is to provide a comprehensive overview of the theme creation process using PHP. This project aims to showcase proficiency in leveraging PHP for dynamic content generation, implementing WordPress best practices, and ensuring a responsive, user-friendly, and well-structured theme. By examining the codebase, design considerations, and integration of essential functionalities, this project aims to demonstrate a thorough understanding of theme development in the WordPress ecosystem.

OUTLINE:

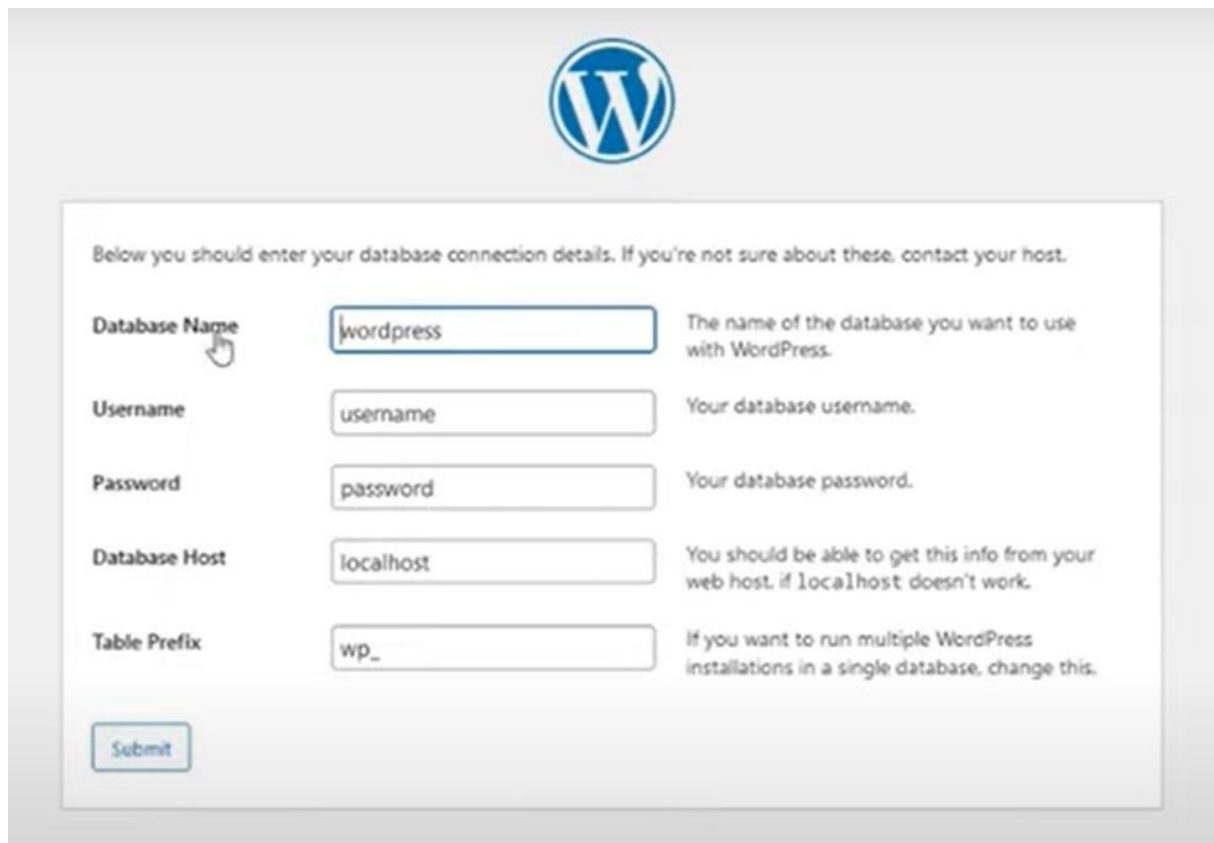
Name	web-project (web-project)
Version	1.0
Author	Aditi
Author website	Undefined
Parent theme	None
Theme features	core-block-patterns, widgets-block-editor, custom-logo, widgets, menus, post-thumbnails
Theme directory location	D:\Xampp\htdocs\wordpress/wp-content/themes/web-project
Auto-updates	Disabled

We have created a theme to implement a blog website. And added WordPress functions and hooks to ensure seamless integration with theme functionality.

Initializing the connection:

To initiate the connection, we first installed the WordPress kit in Xampp\htdocs. Xampp is a web-server solution package in collaboration with Apache. It allows you to connect with MySQL and PhpMyAdmin. Then go to localhost/wordpress.

It then registers the user and database connection details in WordPress.



The image shows the WordPress database connection screen. At the top is the WordPress logo. Below it, a text box says: "Below you should enter your database connection details. If you're not sure about these, contact your host." There are five input fields with labels and descriptions to their right:

Field Label	Input Value	Description
Database Name	wordpress	The name of the database you want to use with WordPress.
Username	username	Your database username.
Password	password	Your database password.
Database Host	localhost	You should be able to get this info from your web host. If localhost doesn't work.
Table Prefix	wp_	If you want to run multiple WordPress installations in a single database, change this.

At the bottom left is a "Submit" button.

Then WordPress is installed.

What is WordPress?

WordPress is a popular and user-friendly content management system (CMS) that allows people to easily create and manage websites. It is open-source, offers themes and plugins for customization, and has a large supportive community. Originally a blogging platform, WordPress has evolved into a versatile CMS used for various types of websites.

Code Implementation:

1. INDEX.PHP

- This serves as the initial landing page when accessing the website. It acts as the homepage of the theme and typically includes a navigation bar, sliders, some posts, a sidebar, and a footer.

Click here to ask Blackbox to help you code faster

```

<?php
    get_header();
    get_template_part('template_parts/nav');
    if(is_home()){
        get_template_part('template_parts/sliders');
    }
?>

<div class="container-fluid">
    <div class="row">
        <div class="col-md-9 p-4 card-group justify-content-end" >
            <?php
                if(have_posts()){
                    while(have_posts()){
                        the_post();
                        if(is_page()){
                            get_template_part('template_parts/page');
                        }else{
                            get_template_part('template_parts/post');
                        }
                    }
                }
            ?>
        </div>
        <!-- Sidebar -->
        <div id = "widget" class="col-md-3 bg-light p-2">
            <?php dynamic_sidebar( 'sidebar-1' ); ?>
        </div>
    </div>
</div>

<!--https://www.billerickson.net/code/wp_query-arguments-->

<!--
<div class="col p-4 card-group justify-content-center">
    <?php /*
    $args = ['s'=> 'hello',//changeable
    'post_type'=>'post',,];

    $myposts = new WP_Query($args);
    if($myposts->have_posts()){
        while($myposts->have_posts()){
            $myposts->the_post();
            get_template_part('template_parts/post'); }
        wp_reset_postdata();*/
    ?>
</div>
-->
<?php get_template_part('template_parts/pagination');?>

<?php get_footer();?>

```

2. NAVIGATION BAR

- Comprising of two files, namely **nav.php**, and **walker.php**, this component facilitates seamless navigation across pages. **walker.php** ensures essential navigation functionalities in

WordPress, while **nav.php** displays elements like the logo, navigation items, and a search bar for specific posts.

Click here to ask Blackbox to help you code faster

```
<nav class="navbar navbar-expand-lg" >
  <div class="container-fluid">
    <?php
      if (has_custom_logo() ) {
        the_custom_logo();
      }else
      {?>
    <a class="navbar-brand site-title" href="<?= get_home_url();?>">
    <?php bloginfo('name')?>
    </a> <?php

  }?>

  <button class="navbar-toggler" type="button" data-bs-toggle="collapse"
  data-bs-target="#navbarSupportedContent" aria-controls="navbarSupportedContent"
  aria-expanded="false" aria-label="Toggle navigation">
    <span class="navbar-toggler-icon"></span>
  </button>

  <div class="collapse navbar-collapse" id="navbarSupportedContent">
    <?php
      wp_nav_menu(
        array(
          'theme_location' => 'header-menu', // Menu location, referenced in Appearance
          'container' => '',
          'menu_class' => 'navbar-nav me-auto mb-2 mb-lg-0',
          'walker' => new header_menu_walker(),));?>

      <form class="d-flex" action="<?php get_home_url();?>">
        <input name="s" class="form-control me-2" type="search" placeholder="Search"
        aria-label="Search">
        <button class="btn btn-outline-success" type="submit">Search</button>
      </form>
    </div>
  </div>
</nav>

</div>
```

```

<?php
1 reference | 0 implementations
class header_menu_walker extends Walker {
    /** ...
    0 references
    public $tree_type = array( 'post_type', 'taxonomy', 'custom' );

    /** ...
    0 references
    public $db_fields = array(
        'parent' => 'menu_item_parent',
        'id'      => 'db_id',
    );

    /** ...
    0 references | 0 overrides
    public function start_lvl( &$output, $depth = 0, $args = null ) { ...
    }

    /** ...
    0 references | 0 overrides
    public function end_lvl( &$output, $depth = 0, $args = null ) { ...
    }

    /** ...
    0 references | 0 overrides
    public function start_el( &$output, $data_object, $depth = 0, $args = null, $current_object_id = 0 ) { ...
    }

    /** ...
    0 references | 0 overrides
    public function end_el( &$output, $data_object, $depth = 0, $args = null ) { ...
    }

    /** ...
    3 references | 0 overrides
    protected function build_atts( $atts = array() ) {
        $attribute_string = '';
        foreach ( $atts as $attr => $value ) {
            if ( false !== $value && '' !== $value && is_scalar( $value ) ) {
                $value
                    = ( 'href' === $attr ) ? esc_url( $value ) : esc_attr( $value );
                $attribute_string .= ' ' . $attr . '=' . $value . ' ';
            }
        }
        return $attribute_string;
    }
}
?>

```

3. SLIDERS.PHP

- Serving as a decorative feature on the homepage, this component consists of three images that slide, controlled by arrow buttons.


```

<?php
$slider_active = get_theme_mod('theme_slider_activate',1);
?>
<?php if($slider_active):?>

<div id="carouselExample" class="carousel slide">
  <div class="carousel-inner">
    <div class="carousel-item active">
      <?php
      $image= get_template_directory_uri().'/assets/images/slider1.jpg';
      if(get_theme_mod('theme_slider_image_1','')!="")
      {
        $image=wp_get_attachment_url(get_theme_mod('theme_slider_image_1',''));
      }
      ?>
      
    </div>
    <div class="carousel-item"> ...
       </div>
    <div class="carousel-item"> ...
       </div>
  </div>
  <button class="carousel-control-prev" type="button" data-bs-target="#carouselExample"
  data-bs-slide="prev">
    <span class="carousel-control-prev-icon" aria-hidden="true"></span>
    <span class="visually-hidden">Previous</span>
  </button>
  <button class="carousel-control-next" type="button" data-bs-target="#carouselExample"
  data-bs-slide="next">
    <span class="carousel-control-next-icon" aria-hidden="true"></span>
    <span class="visually-hidden">Next</span>
  </button>
</div>

<?php endif;?>

```

4. POST.PHP

- In WordPress, posts are typically showcased on the main page of the theme. Users can adjust settings to control the number of posts displayed, and pagination functionality may appear based on the number of posts.

Click here to ask Blackbox to help you code faster

```

<div class="card m-2 shadow border text-center"
style="max-width: 280px;min-width: 200px; display:grid;"
<?php the_post_thumbnail('post-preview')?>
<div class="card-body">
    <h5 class="card-title"><?php the_title()?></h5>
    <p class="card-text"><?= substr(get_the_excerpt(),0,50)?>..</p>
    <a href="<?php the_permalink()?>" class="btn btn-primary">Read more</a>
</div>
</div>

```

5. SINGLEPOST.PHP

- This page displays the full content of a post when users click on it.

```

<div class="text-center justify-content-center">
    <div style="text-align:center">
        <?php
        $post_id = get_the_ID();
        $views = (int)get_post_meta($post_id, 'views', true);
        update_post_meta($post_id, 'views', $views + 1);
        $views = get_post_meta($post_id, 'views', true);
        echo "Views: <i class='fa fa-eye'></i>" . $views . "<br>";
        ?>
        <div class="card-container">
            <?php the_post_thumbnail('post-thumbnail',
            ['style' => 'height:400px;width:500px;display:inline-block;'])?>
            <h5 class="card-title"><?php the_title()?></h5>
            <div class="card-body">
                <?php the_content(); ?>
            </div>
        </div>
    </div>
</div>

```

6. WIDGETS.PHP

- Enabling the integration of customized widgets within WordPress, this component, particularly the sidebar, presents elements like the most popular posts based on view count.

```

<?php
2 references | 0 implementations
class My_Widget extends WP_Widget {
    2 references | 0 overrides
    public function __construct() { ...
    }
    0 references
    public $args = array(
        'before_title' => '<h4 class="widgettitle">',
        'after_title'   => '</h4>',
        'before_widget' => '<div class="widget-wrap">',
        'after_widget'  => '</div></div>',
    );
    0 references | 0 overrides
    public function widget( $args, $instance ) { ...
    }
    0 references | 0 overrides
    public function form( $instance ) {
        $text = ! empty( $instance['text'] ) ? $instance['text'] : esc_html__( '', 'text_domain' );
        $title = ! empty( $instance['title'] ) ? $instance['title'] : esc_html__( '', 'text_domain' );
        $comment = ! empty( $instance['comment'] ) ? $instance['comment'] : esc_html__( '', 'text_domain' );

        <p>
            <label for="<?php echo esc_attr( $this->get_field_id( 'title' ) ); ?>">
                <?php echo esc_html__( 'Title:', 'text_domain' ); ?></label>
                <input class="widefat" id="<?php echo esc_attr( $this->get_field_id( 'title' ) ); ?>"
                    name="<?php echo esc_attr( $this->get_field_name( 'title' ) ); ?>"
                    type="text" value="<?php echo esc_attr( $title ); ?>"
            </p>

        <p> ...
        </p>
        <p> ...
        </p>
    <?php

    0 references | 0 overrides
    public function update( $new_instance, $old_instance ) {
        $instance = array();
        $instance['title'] = ( ! empty( $new_instance['title'] ) ) ? strip_tags( $new_instance['title'] ) : '';
        $instance['text'] = ( ! empty( $new_instance['text'] ) ) ? $new_instance['text'] : '';
        $instance['comment'] = ( ! empty( $new_instance['comment'] ) ) ? $new_instance['comment'] : '';
        return $instance;
    }
}
$my_widget = new My_Widget();

```

7. PAGE.PHP

- This file serves as a template for displaying regular website pages.

```

<div class="m-4 text-center" style="">
    <?php the_post_thumbnail('post-preview')?>
    <div class="card-body">
        <h4 class="card-title"><?php the_title() ?></h4>
        <p class="card-text"><?php the_content() ?>..</p>
    </div>
</div>

```

8. HEADER.PHP

- Containing metadata and necessary HTML tags, this file is included in each file of the theme.

```
<!DOCTYPE html>
<?php ?>
<html <?php language_attributes()??>
<head>
    <meta charset="<?php bloginfo('charset')??">
    <title><?php bloginfo('name')?></title>
    <?php wp_head()??>
</head>
<body <?php body_class()??>
    <?php
        wp_body_open();?>
```

9. FOOTER.PHP

- This component houses the footer tag and includes the footer menu, which is consistent across all pages.

```
<footer class="">
    <div class="container">
        <div class="row">
            </div>
            <div class="col-md-2.5">
                <?php
                    wp_nav_menu(
                        array(
                            'theme_location' => 'footer-menu',
                        )
                    );
                ?>
            </div>
        </div>
    </div>
</footer>
<br>
<?php wp_footer()??>
</body>
</html>
```

10. FUNCTIONS.PHP

- All essential functionalities, such as CSS and JavaScript file inclusions, theme support, navigation menu, and widget registration, as well as custom logo and slider integration, are managed within this file. It plays a crucial role in connecting the WordPress backend with the theme.

```
<?php
//echo get_template_directory_uri().'/assets/css/bootstrap.min.css';

add_action('init',function(){

    //Adding css files, js script, theme support, and actions.

    if (!is_admin()) { ...
    }

    add_theme_support('widgets');
    add_theme_support('menus');
    add_theme_support('post-thumbnails');

    add_image_size('post-preview',270,200,true);
    add_image_size('post-preview-small',100,100,true);
    //register_menu
    register_nav_menus(
        array(
            'header-menu' => 'Header Menu' ,
            'footer-menu' => 'Footer Menu' ,
            'sidebar-menu'=>'Sidebar Menu')
        );
    );
    //custom logo
    add_action('after_setup_theme',function() { ...
    });

    //register sidebars
    add_action('widgets_init',function() { ...
    });

    //SLIDER settings
    add_action( 'customize_register', function ( $wp_customize )
    { ...
    } );

    require get_template_directory() . '/template_parts/walker.php';
    require get_template_directory() . '/template_parts/widgets.php';
```

11. MYSQL DATABASE

- WordPress stores each element within its MySQL database, serving as a log system for all operations carried out within WordPress.

☐	wp_commentmeta	★	Browse	Structure	Search	Insert	Empty	Drop	0	InnoDB	utf8mb4_unicode_520_ci	48.0 KiB	-
☐	wp_comments	★	Browse	Structure	Search	Insert	Empty	Drop	1	InnoDB	utf8mb4_unicode_520_ci	96.0 KiB	-
☐	wp_links	★	Browse	Structure	Search	Insert	Empty	Drop	0	InnoDB	utf8mb4_unicode_520_ci	32.0 KiB	-
☐	wp_options	★	Browse	Structure	Search	Insert	Empty	Drop	182	InnoDB	utf8mb4_unicode_520_ci	2.1 MiB	-
☐	wp_postmeta	★	Browse	Structure	Search	Insert	Empty	Drop	242	InnoDB	utf8mb4_unicode_520_ci	96.0 KiB	-
☐	wp_posts	★	Browse	Structure	Search	Insert	Empty	Drop	95	InnoDB	utf8mb4_unicode_520_ci	176.0 KiB	-
☐	wp_termmeta	★	Browse	Structure	Search	Insert	Empty	Drop	0	InnoDB	utf8mb4_unicode_520_ci	48.0 KiB	-
☐	wp_terms	★	Browse	Structure	Search	Insert	Empty	Drop	4	InnoDB	utf8mb4_unicode_520_ci	48.0 KiB	-
☐	wp_term_relationships	★	Browse	Structure	Search	Insert	Empty	Drop	17	InnoDB	utf8mb4_unicode_520_ci	32.0 KiB	-
☐	wp_term_taxonomy	★	Browse	Structure	Search	Insert	Empty	Drop	4	InnoDB	utf8mb4_unicode_520_ci	48.0 KiB	-
☐	wp_usermeta	★	Browse	Structure	Search	Insert	Empty	Drop	23	InnoDB	utf8mb4_unicode_520_ci	48.0 KiB	-
☐	wp_users	★	Browse	Structure	Search	Insert	Empty	Drop	1	InnoDB	utf8mb4_unicode_520_ci	64.0 KiB	-
12 tables		Sum							569	InnoDB	utf8mb4_general_ci	2.8 MiB	0 B

12. CSS STYLING

- Styling of HTML elements is accomplished through **style.css**, containing various selectors depending on their IDs, classes, and tags.

```

ul#menu-primary-menu-links > li {
    /* Your styles for the list items go here */
    font-weight: bold;
    /* Add more styles as needed */
    display: block;
    list-style-type: none;
    margin-block-start: 1em;
    margin-block-end: 1em;
    margin-inline-start: 0px;
    margin-inline-end: 0px;
    padding-inline-start: 40px;
    color: #57a8e2;
    text-align: center;
}

/* Style for the main navigation */
ul.navbar-nav {
    list-style: none;
    margin: 0;
    padding: 0;
}

/* Style for the main navigation items */
ul.navbar-nav > li {
    display: inline-block;
    position: relative;
}

/* Style for the links in the main navigation */
ul.navbar-nav > li > a {
    display: block;
    padding: 10px 15px;
    text-decoration: none;
    color: #333;
}

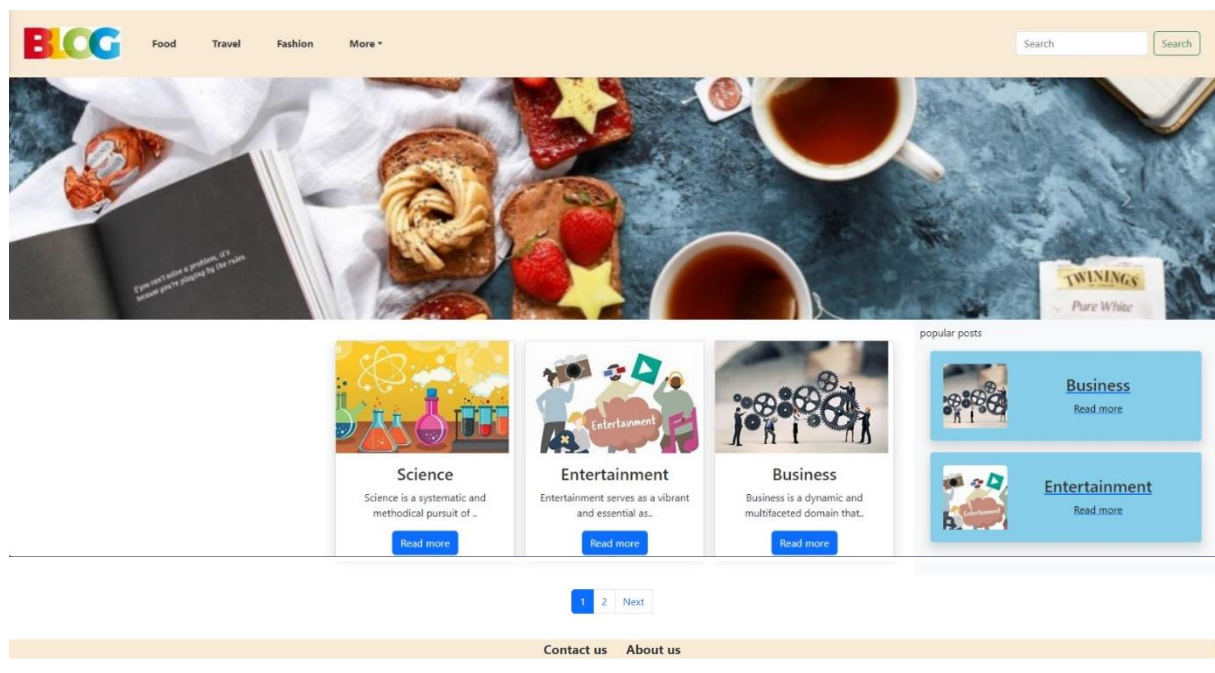
```

Functionalities of the theme:

When you open the homepage, you'll see a menu at the top where you can explore different posts. If you're looking for something specific, you can use the search bar to find posts related to a particular keyword. The homepage also has a slider that adds visual appeal to the theme.

On the right side of the page, there's a widget that shows the most popular posts based on how many times they've been viewed. If there are a lot of posts, you might see pagination at the bottom, which lets you navigate through them easily. You can adjust how many posts you see at once in the WordPress dashboard settings.

At the bottom of the page, there's a footer menu. You can change what's in this menu through the WordPress dashboard as well.



When you click on any post, you'll see more details about it. This includes the title, a thumbnail image, and the content of the post. You'll also see how many times the post has been viewed.



Science

Science is a systematic and methodical pursuit of knowledge that seeks to understand the natural world through observation, experimentation, and analysis. It is a dynamic and ever-evolving field that encompasses a wide range of disciplines, including physics, chemistry, biology, astronomy, and more. The scientific method, a fundamental approach in science, involves formulating hypotheses, conducting experiments, and drawing conclusions based on empirical evidence. Science has played a crucial role in shaping our understanding of the universe, from unraveling the mysteries of the smallest particles to exploring the vastness of outer space. It has led to groundbreaking discoveries, technological advancements, and improvements in various aspects of human life, from medicine and agriculture to communication and transportation. Science also fosters critical thinking, skepticism, and a spirit of inquiry,

CONCLUSION:

Through this project, it becomes evident that even creating a basic theme involves significant backend work. It provides a learning opportunity about WordPress functionalities and the utilization of languages such as HTML, CSS, and PHP.

REFERENCES:

- YouTube tutorials on WordPress theme development.
- [Developer.wordpress.org](https://developer.wordpress.org)
- Bootstrap documentation
- [W3schools.com](https://www.w3schools.com)